

Autonomous Research Paper Summarizer

Complete Algorithm Specification, Pipeline Workflow & Technical Modules

SYSTEM ALGORITHM & DATA WORKFLOW

1 Document Ingestion & Text Extraction

STAGE 1 • PDF PARSING LAYER

Extract raw text and multi-column document structure from uploaded PDF files using tools like PyMuPDF (fitz) or pdfplumber. Handle multi-column layouts, clean artifact characters, and filter out running headers, footers, and page numbers.

2 Text Preprocessing & Semantic Chunking

STAGE 2 • TOKEN OPTIMIZATION

Clean special characters and split the extracted raw text into manageable text chunks using RecursiveCharacterTextSplitter (e.g., 2000 characters per chunk with a 200 character overlap) to ensure context retention across chunk boundaries while respecting LLM context window limits.

3 Embedding & Vector Store Indexing

STAGE 3 • RETRIEVAL LAYER

Convert text chunks into dense numeric vector embeddings using model embeddings (e.g., OpenAI text-embedding-3-small or HuggingFace Sentence Transformers) and store them in a local vector database (FAISS / ChromaDB) to power Retrieval-Augmented Generation (RAG).

4 Autonomous Map-Reduce Summarization

STAGE 4 • LLM EXECUTION CHAIN

Execute a Map-Reduce summarization chain via LangChain. The "Map phase" generates localized summaries of each text chunk, while the "Reduce phase" synthesizes these into a structured report covering Problem Statement, Methodology, Findings, and Limitations.

5 Interactive UI & Document Chatbot

STAGE 5 • USER INTERFACE

Display the finalized structured executive summary on an interactive Streamlit UI while enabling a secondary conversational RAG Q&A interface over the vector database.

TECHNICAL SPECIFICATIONS & COMPONENT DETAILS

Core Technologies

- **Language:** Python 3.10+
- **PDF Parsing:** PyMuPDF (fitz), pdfplumber
- **Orchestration:** LangChain Framework
- **Vector Database:** FAISS / ChromaDB
- **LLMs:** OpenAI GPT-3.5/GPT-4 or Ollama
- **Frontend:** Streamlit Web Framework

Key Performance Advantages

- **Context Preservation:** Overlapping chunk boundaries eliminate loss of information across sections.
- **Token Efficiency:** Map-Reduce technique permits summarization of 50+ page papers safely.
- **Interactive QA:** Embedded vector index allows instant follow-up questions via RAG.
- **Structured Output:** Standardized Markdown summaries simplify paper comparison.